CS 314H Honors Data Structures — Fall 2026	Programming Assignment #6
Bounding Volume Hierarchies	Due November 11/14/26, 2026

In this assignment you will work **individually** to implement a data structure known as a bounding volume hierarchy (BVH). A key challenge in this assignment will be to carefully and thoroughly test your data structure, so you will also be asked to design a testing program for your code.

You will again be writing Peer Reviews, with the following schedule.

Program and Tests Due	11:59 pm Sunday November 8
Peer Reviews Due	11:59 pm Wednesday November 11
Testing Report and Final Code Due	11:59 pm Saturday November 14

1 Bounding Volume Hierarchies

A *bounding volume hierarchy* (BVH) is a binary tree that enables the efficient of search for geometric shapes within some larger space. For example, BVH's are commonly used in ray tracing and other graphics applications to quickly identify the geometric objects in a scene that a ray of light will intersect. While BVH's typically describe a 3D space of objects, we will in the assignment work with a 2D space of objects.

Each *internal node* of a BVH stores a *bounding box* for the subtree rooted at that node. For a 2D space, this bounding box is the smallest rectangle that contains all of the objects in the subtree. Each *leaf node* of a BVH stores one geometric object, which for this assignment will be a triangle.

Figure 2 shows an example of a 2D BVH, where each bounding box is represented by two Cartesian points, its minimum and maximum positions, and the object in each leaf is represented by three Cartesian points. Notice that the BVH is a balanced tree but that the bounding boxes of sibling nodes can overlap, so searching for a specific area might involve the traversal of both subtrees.

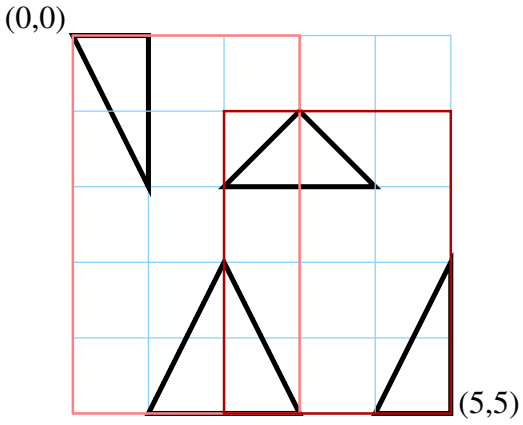


Figure 1: Four triangles in a 2D space, and the bounding boxes that would arise from splitting along the x-axis, where the left bounding box is the half-space that contains the two left-most triangles, while the right bounding box is the half-space that contains the two right-most triangles.

BVH's are typically built from a set of geometric objects, as opposed to being incrementally constructed by adding one object one at a time. A common algorithm for building a BVH divides the space by a plane along one of the axes, such that each side of the plane contains all of the objects whose centers reside on one side of the plane. The bounding box for each half of the space is then created by finding the minimum and maximum extents of the objects on that half of the space.

The algorithm repeats the process of dividing each of the half-spaces until a half-space contains just a single object. However, the axis that is used to divide the space will alternate for each level down the tree. Thus for two dimensions, the odd levels will divide along one axis, say the x-axis, while the even levels will divide along the other axis.

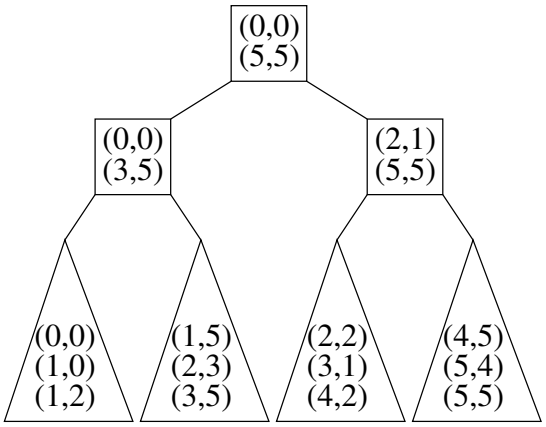


Figure 2: A BVH for objects shown in Figure 1. Each bounding box is represented by its upper left and lower right corners, and each triangle is represented by the coordinates that define the triangle.

A key design decision for BVHs is method of determining where to divide the space. One common approach uses the midrange, which is the average of the minimum and maximum coordinates for all objects in the space that is being divided. Equation 1 shows how the line to split on can be calculated. A second common approach uses the median value of the centers of all objects in the space that is being divided. This guarantees a balanced split of triangles.

$$\text{Midrange on Axis} = \frac{\text{Minimum Point on Axis} + \text{Maximum Point on Axis}}{2} \tag{1}$$

An important operation on BVH's is the **intersectsRay** operation, which identifies the object that a ray will intersect. The inputs to this operation are the ray's source and the direction that it travels. The output is the first object in the space that would be hit. A naive approach would check every object within the BVH, but a simple optimization recognizes that an object cannot intersect the ray if the object's bounding box does not intersect the ray. With this insight, a faster algorithm performs the following starting with the root. If the current node is a leaf, it checks the geometry for intersection and returns. If the current node is not a leaf, it checks to see if the ray intersects the current node's bounding box; on intersection, recurse, testing intersection on each child; otherwise return no intersections.

2 Your Assignment

Your assignment consists of three parts, one that builds a BVH, one that visualizes a BVH, and a third that tests a BVH.

2.1 Building a BVH

Implement a 2D BVH that stores triangles as objects. (Triangles are one of the most common shapes used in graphics.) In particular you should implement the following interface:

```
package assignment;
public interface BVH {
       void setSplitMethod(SplitMethod splitMethod);
       void buildBVH(List<Shape> shapeList);
       void insert(Shape shape);
       void remove(Shape shape);
       Set<Shape> findCollision(Point point);
       Shape intersectRay (Point origin, Vector2D direction);
       String toString();
}
```

The buildBVH method. Build a BVH tree using the split method specified by `setSplitMethod()` or the median if no split method is set. The algorithm to perform splits is described in Section 1. If the centroid of a triangle lies along the line you're splitting on, you should break the tie by attempting to balance the number of triangles on each side of the split. If, after this process, one side of the split still has no triangles, you should isolate the triangle whose centroid is nearest to the split and group the others into a separate bounding box.

The insert method. A BVH is traditionally completely rebuilt after objects have been added or removed, but a complete rebuild is expensive if you expect frequent insertions and removals. For this assignment, you should structure your BVH with properties similar to an AVL tree, that is, for every node, the height of its left and right subtrees differs by at most 1. To guarantee the tree is balanced after an AVL operation, you may require buildBVH to generate a balanced tree.

The `insert()` method should first traverse the BVH to find an ideal location to insert the new triangle. For each parent node, your BVH should select which child to insert a new triangle into according to the following priorities (in decreasing order of significance):

1. Children that are leaves should be given a higher priority
2. Children whose bounding box would grow less in area (before rotations) should be given a higher priority
3. Children whose height is less should be given higher priority
4. If everything else is equal you should pick the child randomly

After finding an ideal location to place the triangle, you should rebalance the tree to restore the AVL tree structure property.

The remove method. The `remove()` method removes the given triangle and, if necessary, rebalances the tree to restore the AVL structure property.

The findCollision method. Another use of BVHs is to detect collisions among objects. The `findCollision()` method returns a set of triangles containing all objects in the BVH that intersect the provided point.

The intersectsRay method. The `intersectsRay()` method returns the first triangle that intersects a ray with the given origin and the given direction. The direction is given as a vector, and your code should use the algorithm described in Section 1.

The toString method. The `toString()` method returns a string that represents the pre-order traversal of the BVH. Each node of the BVH is denoted by a line in the returned string that starts with a number of tabs equal to the node's depth within the BVH, where the root lies at depth 0 and has no tabs. An internal node will be formatted as, "[($\min_X$, $\min_Y$), ($\max_X$, $\max_Y$)]," while a leaf will be formatted as "(a_X, a_Y), (b_X, b_Y), (c_X, c_Y)." For example, the BVH in Figure 2 would output the following for the toString method:

```
[(0, 0), (5, 5)]
    [(0, 0), (3, 5)]
        (0, 0) (1, 0) (1, 2)
        (1, 5) (2, 3) (3, 5)
    [(2, 1), (5, 5)]
        (2, 2) (3, 1) (4, 2)
        (4, 5) (5, 3) (5, 5)
```

2.2 Visualizing the BVH

To visualize your BVH, you should implement the `Visualization` class, which is responsible for managing a simple GUI that will display triangles visible from a certain point. The GUI is a 400 $\times$ 400 pixel frame with a 21 $\times$ 21 square in the middle. This square acts as an entity and doesn't leave the middle of the screen. Instead, the triangles surrounding the square should be shifted to act like the square is moving. The Visualization class is also responsible for using a Bounded Volume Hierarchy to calculate the pixels within line of sight of the square. The interface for the `Visualization` class is below.

To help you implement this function, you should consider that there are two different coordinate planes. There is a GUI coordinate plane that contains the pixels visible and there is a virtual coordinate plane which contains the triangles you should display. Consider the example in Figure 3. Here you have the GUI coordinate plane which goes from (0,0) in the top left to (400,400) in the bottom right. No matter what input you provide, these coordinates are fixed. However, the locations of triangles are provided as virtual coordinates and you must translate those to GUI coordinates. This involves applying some shift operation given the current position of the square in the virtual coordinate space. In the example provided, the bottom left of the triangle is at virtual coordinate (0,100) and physical coordinate (200,300) when the square's center is at virtual coordinate (0,0).

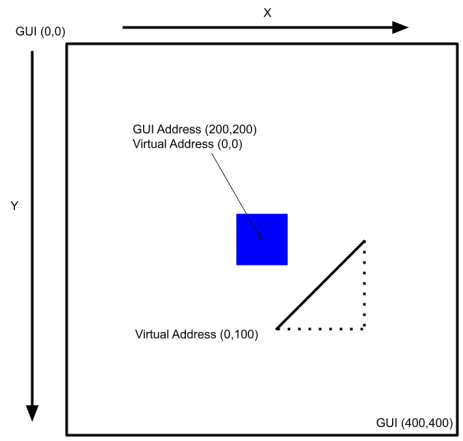


Figure 3: Default Visualization with Triangle

```
package assignment;
public class Visualization {
       boolean moveSquare(char keyPressed);
       LinkedList<Point> drawGUI();
}
```

The moveSquare method. The `moveSquare()` method accepts as input a key press (WASD) and moves the square within the virtual coordinate plane. Since the square will remain stationary within the pixel frame, the square should act like it's moving in the specified direction by shifting the surrounding environment. If the square's movement would intersect a triangle, the square should remain in its current position. Each input should move the square in the virtual plane by 5 units.

The drawGUI method. Given the current position of the square, the `drawGUI()` method returns an LinkedList of pixels that need to be painted. These pixels would represent all the pixels seen by the observer (the center of the square). In Figure 3 this would be solid line of the triangle.

2.3 Testing Your Code

Create a test suite to test the correctness of your BVH. You may use JUnit or just write a program that manually runs the appropriate tests.

Your test suite should work with any implementation of the BVH interface. It should not find any errors in a correct implementation. For errors that it does find, it should produce output that would be useful to a human. You are not required to test the portions that you do not implement yourself, but you should test everything that you do implement.

3 Karma

This project makes several simplifying assumptions that you could relax.

- We only use triangles as objects, but you could support other types of objects.
- Our specification of a BVH uses what's called an AABB box for each bounding box, but these AABB boxes can perform really poorly in some cases. (Can you identify the cases where performance will suffer?) Try shapes other than axis-aligned boxes to provide tighter bounding shapes.
- Our project operates in the 2D plane, but you could try supporting objects in a 3D space.
- We have described a simple way to implement `SplitMethod()`, but you could instead implement the most common approach, which splits the space based on the Crofton formula, which splits the space in a way that reduces the probability that a ray will intersect with the bounding box and not collide with its children. Feel free to do research on the Crofton formula if you want to try this idea.
- For animations, a useful feature is to update the BVH as triangles move over time. The rotations described in this assignment have the goal of balancing the tree, i.e., minimizing tree depth, but the quality of a BVH is typically defined in terms of the surface area of the bounding boxes: A smaller surface area will minimize the number of times that a ray will intersect a bounding box without intersecting any of the enclosed objects. To instead preserve the quality of the BVH over time as leaves move, you can perform a different type of rotation, as described in the following paper: https://hwrt.cs.utah.edu/papers/hwrt_rotations.pdf.
- Implement a ray tracer that uses your BVH to detect object intersections. A ray tracer is a rendering technique that can produce photo-realistic effects by modeling the transport of light rays that bounce around a scene. The algorithm actually works backwards by tracing rays from some imaginary eye into the scene and computing the value of the ray's color as it is reflected and refracted through the scene. This backwards approach is efficient because there are many rays that do not hit the eye do not need to be computed.

4 What to Turn in

Except for the specific names of the files, the submission directions are the same as for Assignment 5, but we repeat them below for your convenience.

4.1 Deadline 1

Submit just your code. *You may receive bonus points if you submit largely functional code by this deadline.* If your submission is late, your reviewers may choose whether they wish to review your code, so it is in your best interest to not take any late days for this deadline. Regardless of whether you submit your assignment on time, you will be assigned several projects to review.

Turn in `BoundedVolumeHierarchy.java`, `Visualization.java`, and any other files necessary to build and run your implementation. Be sure to comment all classes, methods, and fields, and any non-obvious portions of your assignment. Your code should be packaged in a `src` directory and all of your classes should be in the `assignment` package. Make sure that your code compiles and that all classes have the correct names and are in the correct files.

4.2 Deadline 2

Your peer reviews are due. If you turn these in late, your grade will be penalized.

4.3 Deadline 3

Your report, your revised code (optional), testing report, and reviewer grades are due. You have the option to fix your code based on the feedback you received. The testing report should cover any insights gained from the review process, including what you did, how you did it, what you learned, what you learned from the reviews of your own code, what this revealed about your code, and what changes you made or would make if you had more time. **Your final report should also cover time complexity of the functions implemented.**

Acknowledgments. This assignment was created by Christopher Hill and Calvin Lin. We thank Annie Hu, John Jennings, Ojas Phirke, Arnav Sastry, Dillon Walker, and Etienne Vouga for their helpful comments for improving this assignment.